ESCUELA POLITÉCNICA NACIONAL

Facultad de Ingeniería de Sistemas · Carrera de Ciencia de Datos e Inteligencia Artificial

INFORME DE PRÁCTICA N.º 1

Consolidación de reportes de ventas

Tópico 1 del PEA · Asignatura IDSD422 — Estadística y Programación para Ciencia de Datos I

Estudiante(s)	Jácome Viracocha Johan David
Docente	Ing. Christian Suarez
Período académico	2026-B
Fecha de entrega	19-10-2026
Repositorio (fork)	https://github.com/Daxin7/IDSD422-lab1-lectura-datos.git

1· Introducción

RetailAndina S.A. es una cadena ecuatoriana de tiendas de consumo masivo que recibe semanalmente información de ventas proveniente de diferentes sistemas. En esta práctica se trabajó con tres archivos que representan el mismo reporte: un archivo CSV generado por el sistema ERP de las sucursales, un archivo JSON proveniente de la API de la plataforma de fidelización y un archivo Excel correspondiente al consolidado elaborado por el área de Finanzas. Debido a que la información proviene de fuentes y formatos distintos, es necesario verificar que los datos sean consistentes antes de utilizarlos para el análisis.

2 · Objetivos

Objetivo general

Analizar y comparar un mismo reporte semanal de ventas de RetailAndina S.A. almacenado en formatos CSV, JSON y Excel, utilizando Python y R para verificar la consistencia de la información y obtener indicadores iniciales que apoyen la toma de decisiones.

Objetivos específicos

- Comparar la estructura de los tres reportes mediante sus dimensiones, nombres de las columnas y tipos de datos inferidos, con el fin de comprobar que contienen la misma información.

- Calcular los principales indicadores solicitados para el análisis; venta promedio por sucursal, ticket promedio, sucursal con mayor venta, sucursal con menor venta y región con mayor venta total.

- Leer los archivos ventas.csv , ventas.json y ventas.xlsx utilizando herramientas de Python y R, verificando que los tres formatos puedan ser cargados correctamente para su análisis.

3 · Desarrollo de la práctica

3.1 leer_excel.py

En este paso resolví el script de leer_excel.py dentro del entorno de Google Colab completando los TODOs; Primero importé la librería de pandas, luego cargue los datos en una carpeta con el nombre “data” y leí el archivo con la función de pandas “pd.read_excel()”. Para finalizar imprimí las dimensiones y las primeras filas del DataFrame.

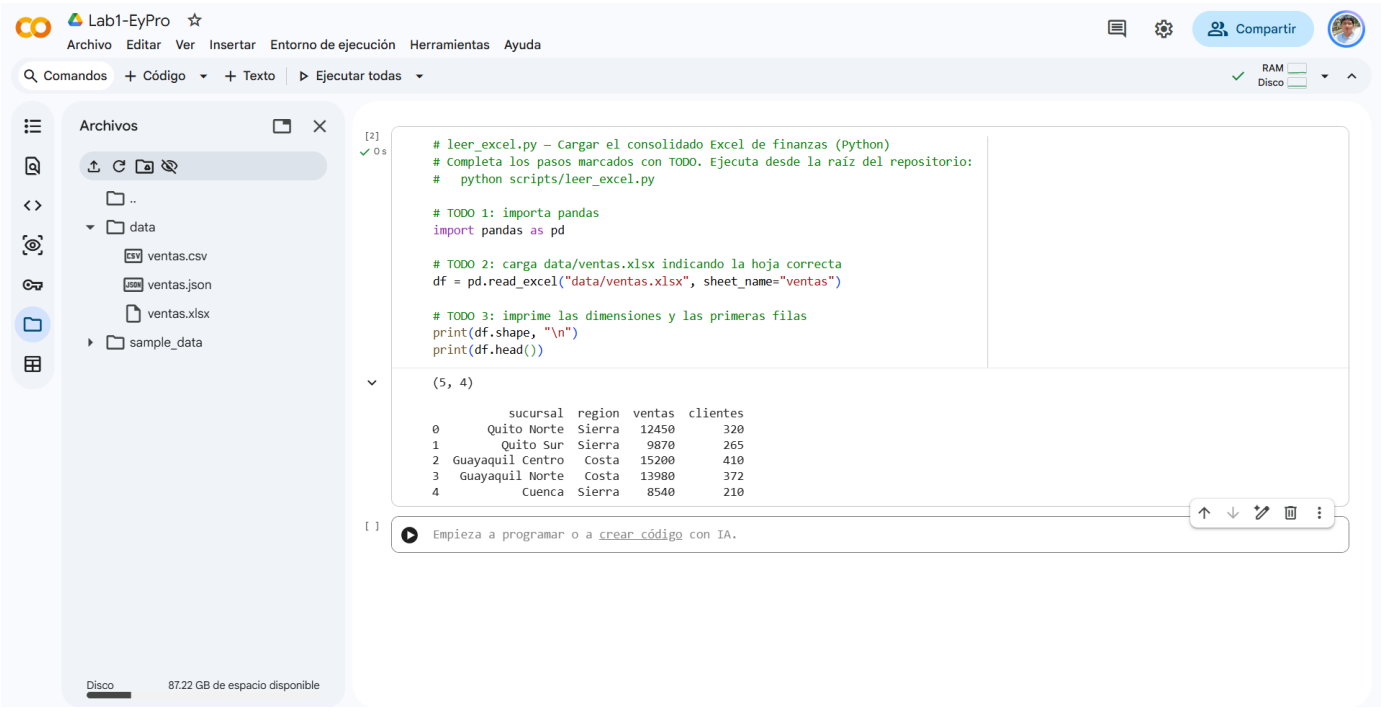


Figura 1: resolución leer_excel.py 1

Código

```
# TODO 1: importa pandas
import pandas as pd

# TODO 2: carga data/ventas.xlsx indicando la hoja correcta
df = pd.read_excel("data/ventas.xlsx", sheet_name="ventas")

# TODO 3: imprime las dimensiones y las primeras filas
print(df.shape, "\n")
print(df.head())
```

3.2 leer_excel.R

En este paso resolví el script de leer_excel.R dentro del entorno de Posit Cloud completando los TODOs; Primero importé el paquete que me permite trabajar con archivos Excel en R para luego invocar la librería, luego lo convertí a un DataFrame para poder manejarlo con facilidad, imprimí las dimensiones del DataFrame con la función “dim()” y las primeras filas con la función “head()”.

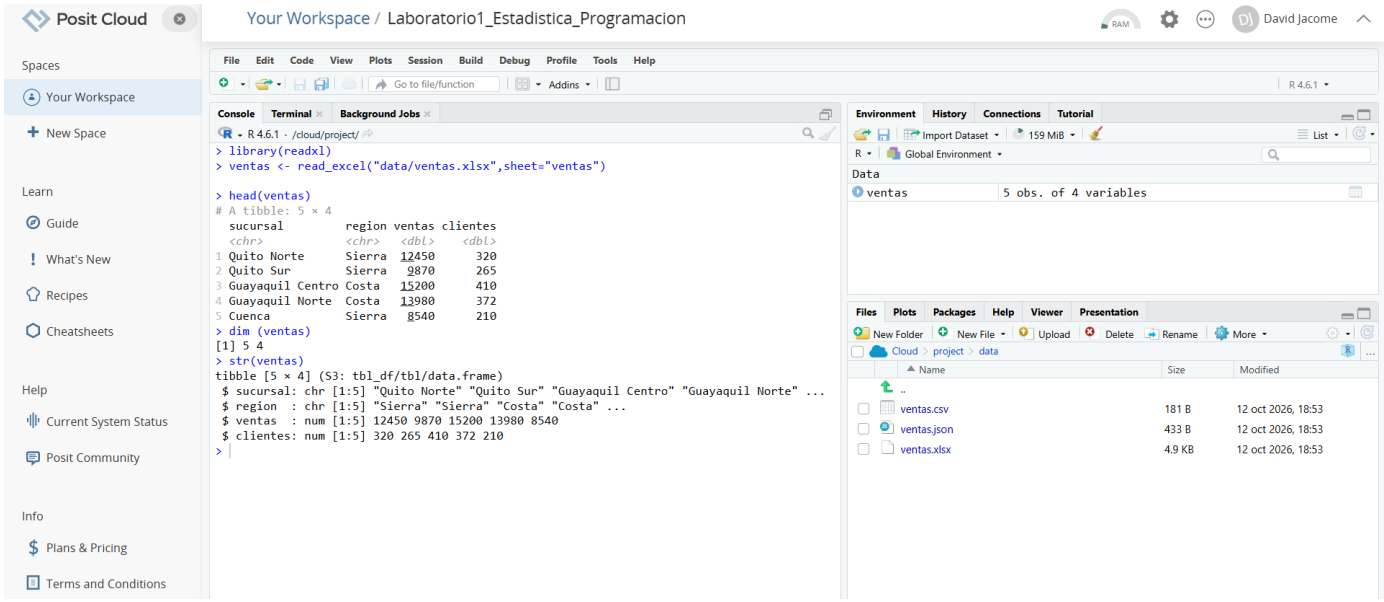


Figura 2: resolución leer_excel.R

Código

```
# TRABAJO COMPLETADO
install.packages("readxl")
library(readxl)
ventas <- read_excel("data/ventas.xlsx",sheet="ventas")
head(ventas)
dim (ventas)
str(ventas)
```

3.3 leer_json.R

En este paso resolví el script de leer_json.R dentro del entorno de Posit Cloud completando los TODOs; Primero importé el paquete que me permite trabajar con archivo json en R para luego invocar la librería, luego lo convertí a un dataframe para poder manejarlo con facilidad, imprimí las dimensiones del dataframe con la función “dim()” y las primeras filas con la función “head()”.

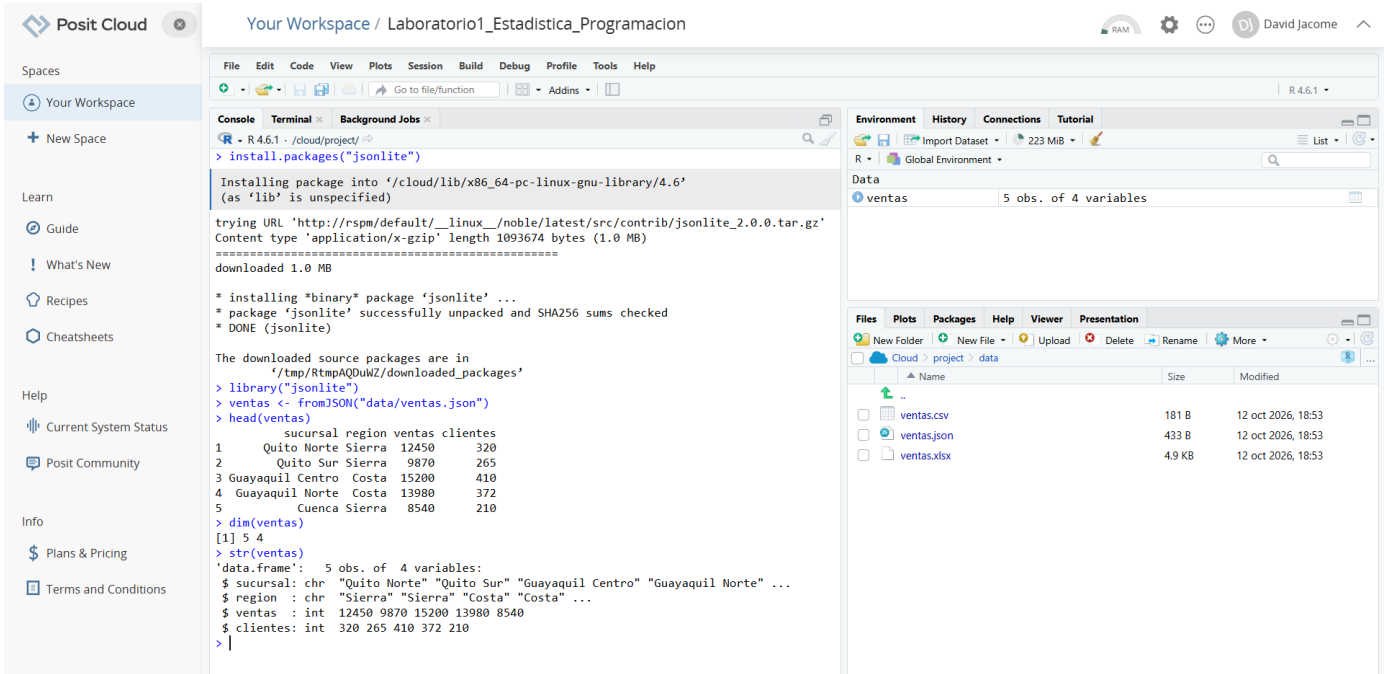


Figura 3: resolución leer_json.R

Código

```
# TRABAJO REALIZADO
install.packages("jsonlite") # paquete que mepremite trabajar con archivos de formato json
library("jsonlite") # invocar la libreria
ventas <- fromJSON("data/ventas.json") # leer el archivo
head(ventas)
dim (ventas)
str(ventas)
```

3.4 leer_json.py

En este paso resolví el script de leer_json.py dentro del entorno de Google colab completando los TODOs; Primero importé la librería de pandas y json, luego cargué los datos con “json.load (f)”, luego los normalicé para un dataframe con la función “json_normalize”. Finalmente imprimí las dimensiones del dataframe y las primeras filas.

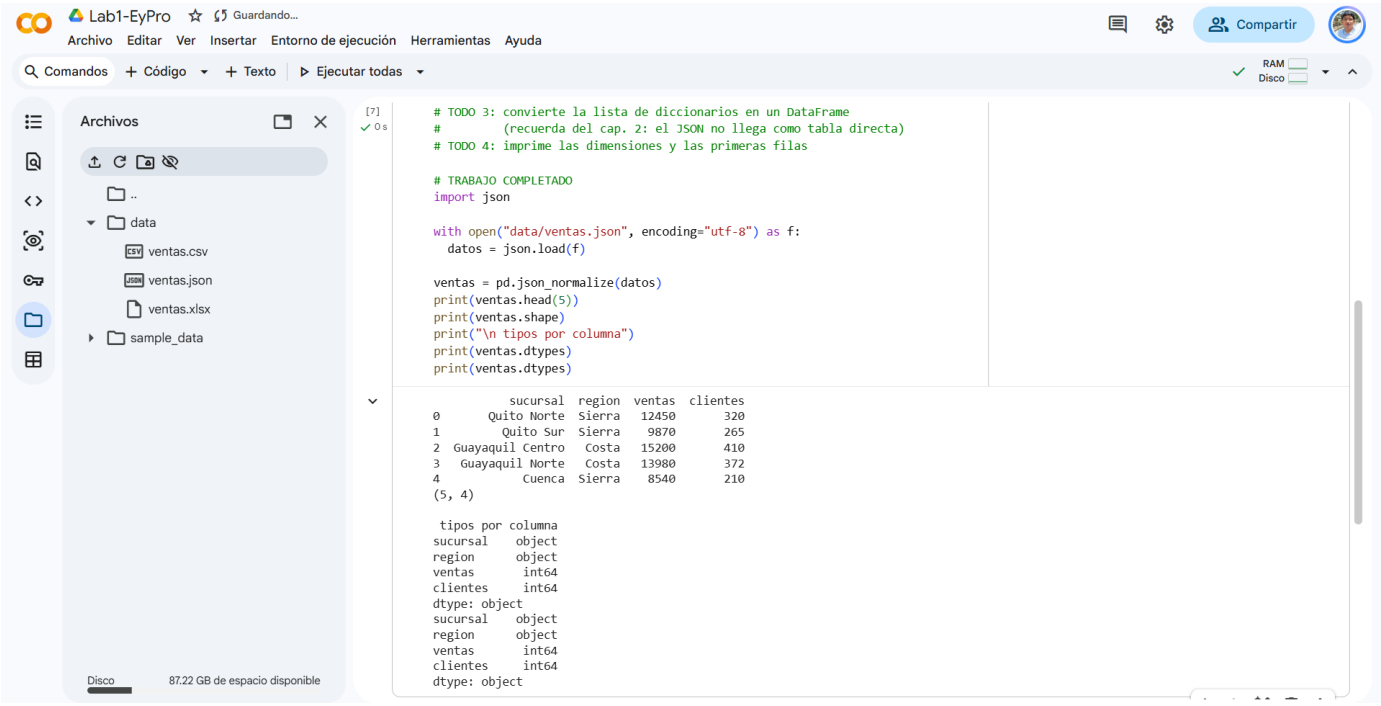


Figura 4: resolución leer_json.py

Código

```
# TRABAJO COMPLETADO
import pandas as pd
import json

with open("data/ventas.json", encoding="utf-8") as f:
  datos = json.load(f)

ventas = pd.json_normalize(datos)
print(ventas.head(5))
print(ventas.shape)
print("\n tipos por columna")
print(ventas.dtypes)
print(ventas.dtypes)
```

3.5 leer_csv.R

En este paso resolví el script de leer_csv.R dentro del entorno de Posit Cloud completando los TODOs; Primero importé la librería de “readr”, luego cargué los datos con “read_csv”, luego imprimí las primeras filas con “head(ventas)” y finalmente imprimí las dimensiones del dataframe.

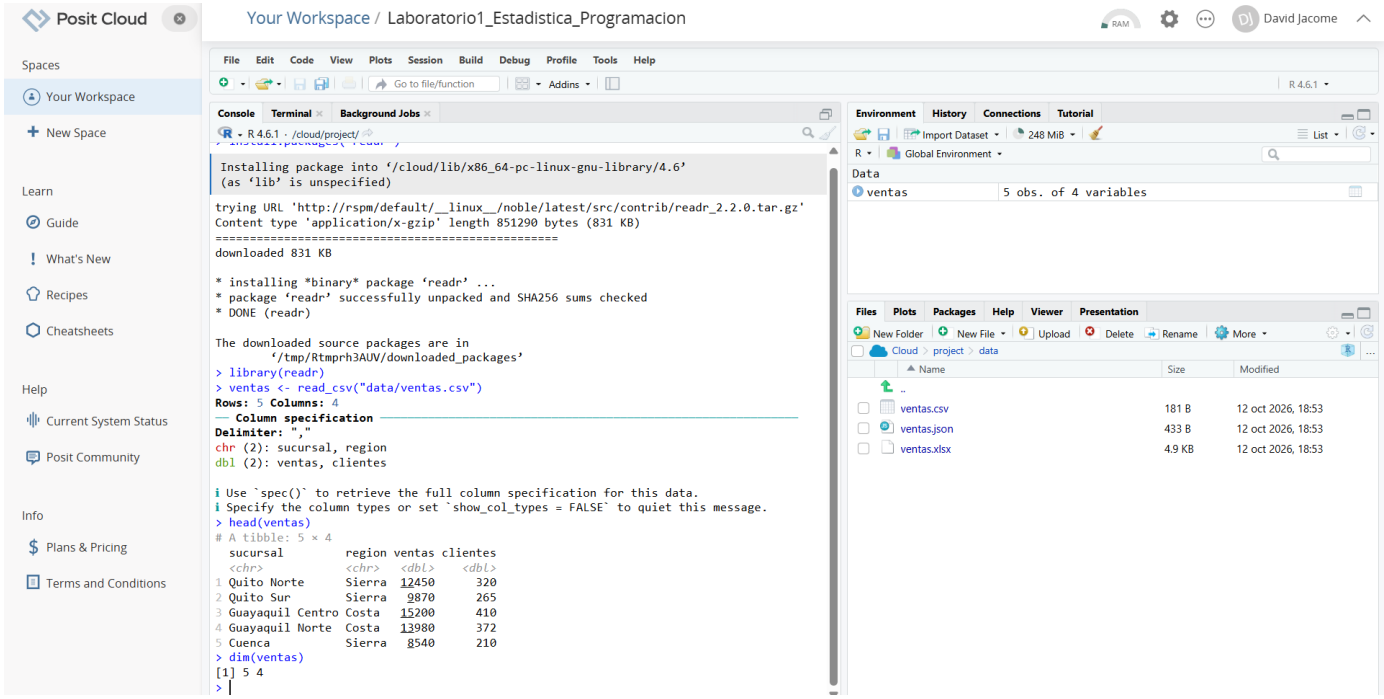


Figura 5: resolución leer_csv.R

Código

```
# TRABAJO REALIZADO
install.packages("readr") # cargar la libreria
library(readr) # invocar el paquete
ventas <- read_csv("data/ventas.csv") # leer el archivo
head(ventas)
dim(ventas) # para ver las dimensiones del dataframe
str(ventas)
```

3.6 leer_csv.py

En este paso resolví el script de leer_csv.py dentro del entorno de Google Cloud completando los TODOs; Primero importé la librería de pandas, luego cargué los datos como un dataframe con la función “.read_csv("data/ventas.csv")”, luego imprimí las primeras filas con “.head()” y finalmente imprimí las dimensiones del dataframe con la función

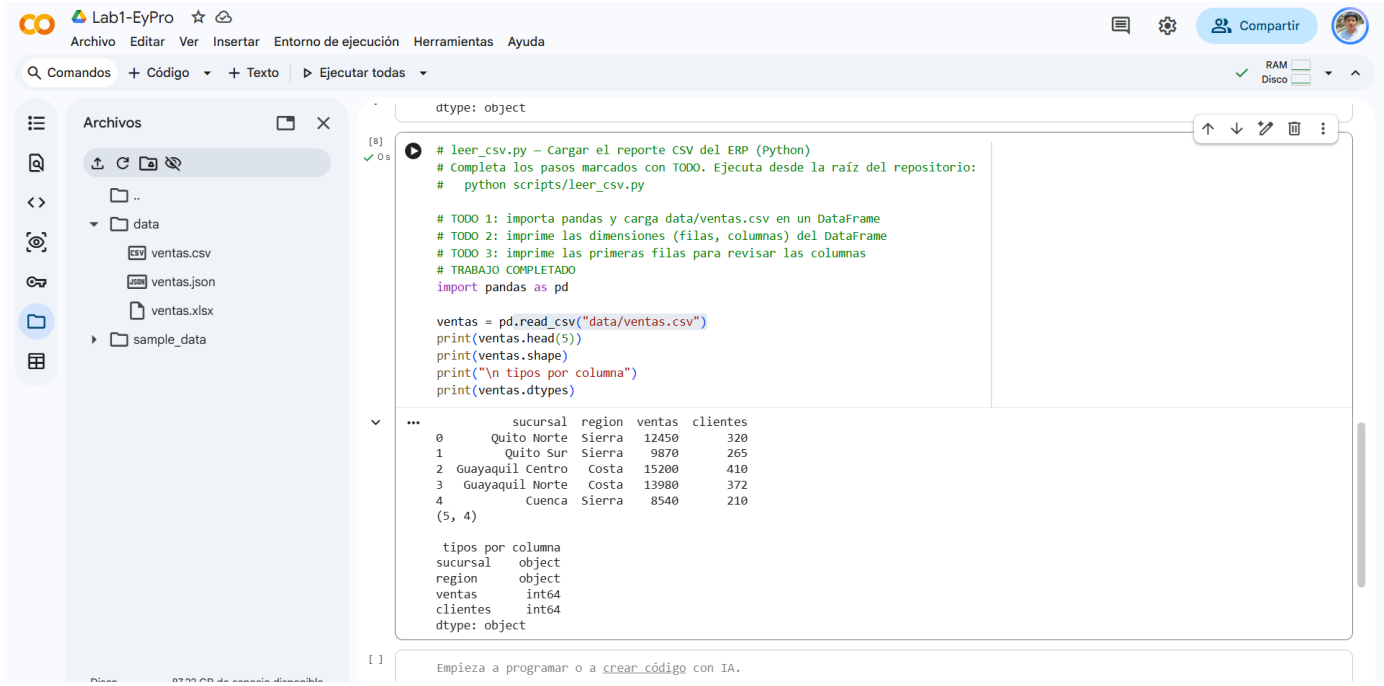


Figura 6: resolución leer_csv.py

Código

```
# TRABAJO COMPLETADO
import pandas as pd

ventas = pd.read_csv("data/ventas.csv")
print(ventas.head(5))
print(ventas.shape)
print("\n tipos por columna")
print(ventas.dtypes)
```

3.7 inspeccion.py

En este paso resolví el script de inspeccion.py dentro del entorno de Google Cloud completando los TODOs; Primero importé la librería de pandas, luego creé la columna derivada "ticket" = ventas / clientes. Luego imprimí la venta promedio por sucursal, el ticket promedio, la sucursal con mayor y menor venta y finalmente la región con mayor venta total.

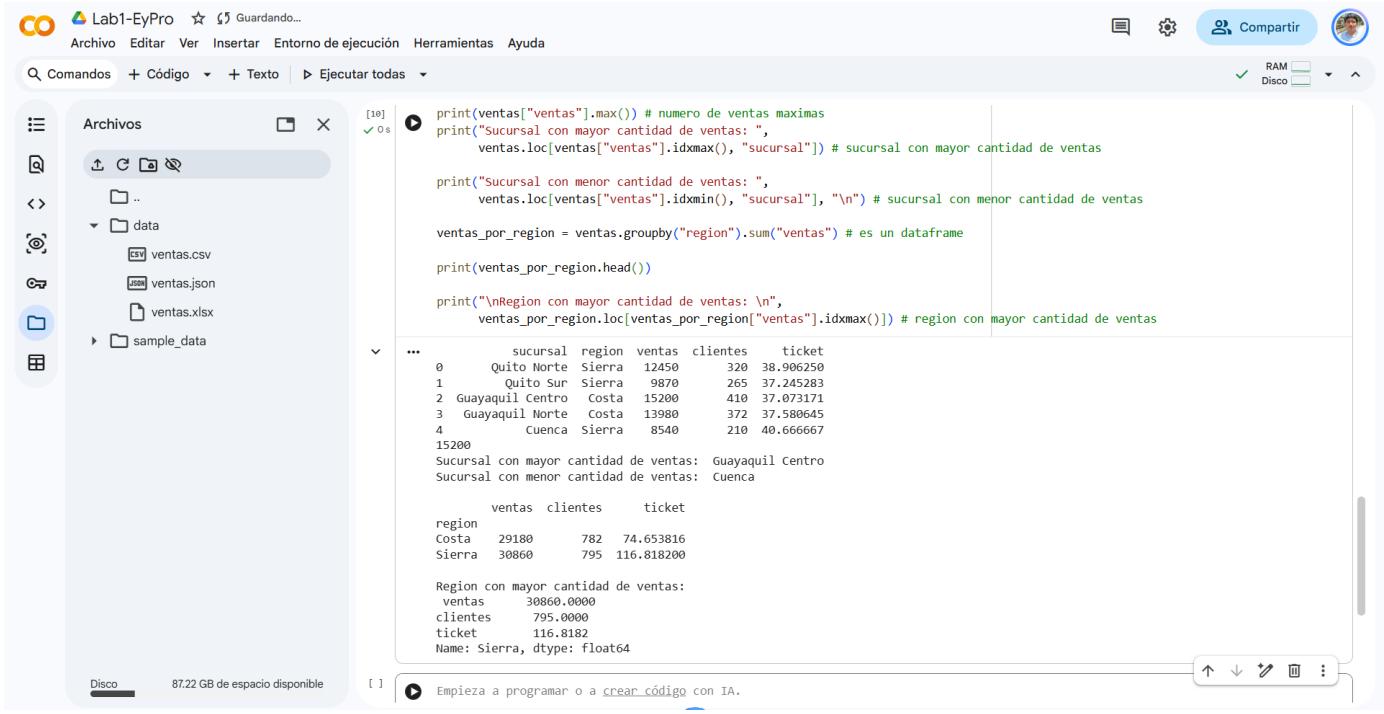


Figura 7: resolución inspeccion.py

Código

```
# TRABAJO COMPLETADO
import pandas as pd
ventas = pd.read_csv("data/ventas.csv")
ventas["ticket"] = ventas["ventas"] / ventas["clientes"]
print(ventas.head())

# =======
print(ventas["ventas"].max()) # numero de ventas maximas
print("Sucursal con mayor cantidad de ventas: ",
    ventas.loc[ventas["ventas"].idxmax(), "sucursal"]) # sucursal con mayor cantidad de ventas

print("Sucursal con menor cantidad de ventas: ",
    ventas.loc[ventas["ventas"].idxmin(), "sucursal"], "\n") # sucursal con menor cantidad de
ventas

ventas_por_region = ventas.groupby("region").sum("ventas") # es un dataframe

print(ventas_por_region.head())

print("\nRegion con mayor cantidad de ventas: \n",
    ventas_por_region.loc[ventas_por_region["ventas"].idxmax()]) # region con mayor cantidad
de ventas
```

3.8 inspeccion.R

En este paso resolví el script de inspeccion.R dentro del entorno de Posit Cloud completando los TODOs; Primero importé la librería de pandas, luego creé la columna derivada "ticket" = ventas / clientes. Luego imprimí la venta promedio por sucursal, el ticket promedio, la sucursal con mayor y menor venta y finalmente la región con mayor venta total.

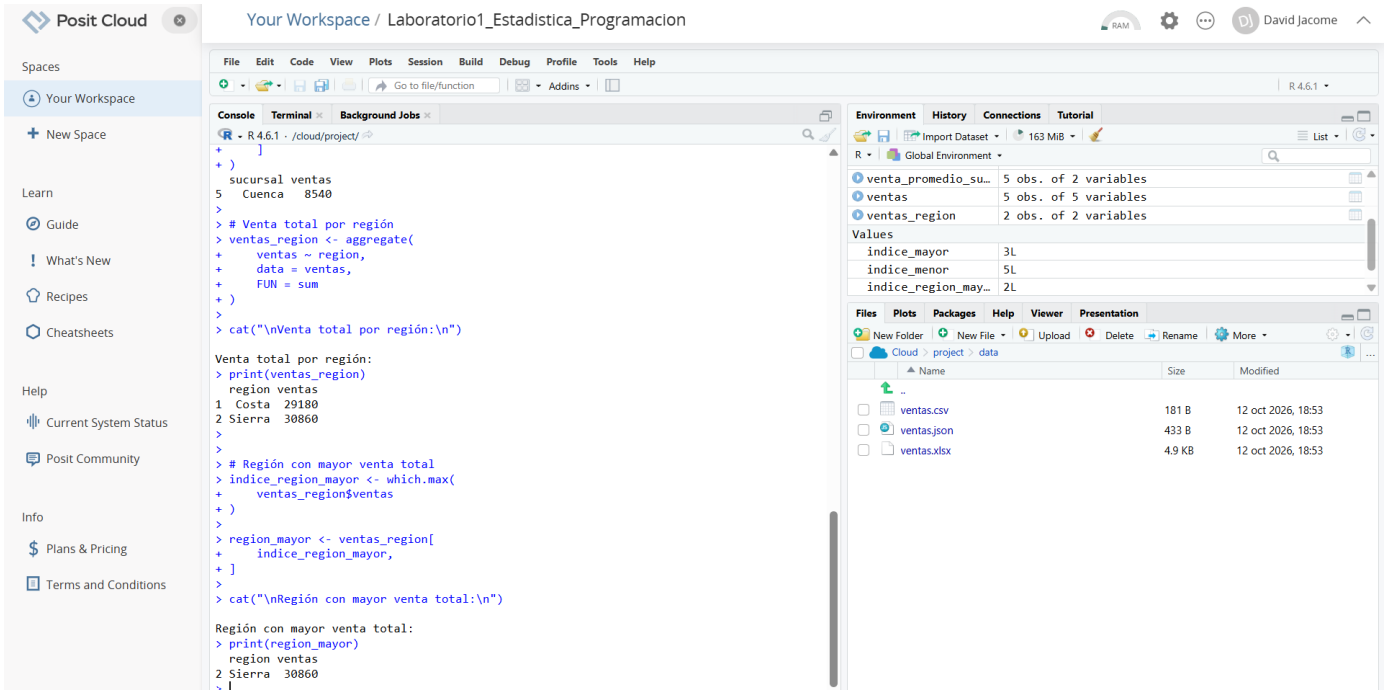


Figura 8: resolución inspeccion.R

Código

```
# TODO 1: carga data/ventas.csv
ventas <- read.csv("data/ventas.csv", stringsAsFactors = FALSE)
# TODO 2: imprime la estructura (str) para ver los tipos
#         (fija si region llegó como character o factor)
str(ventas)
# TODO 3: crea la columna derivada "ticket" = ventas / clientes
ventas$ticket <- ventas$ventas / ventas$clientes
# TODO 4: calcula e imprime:
#         - venta promedio por sucursal (aggregate o dplyr)
# Venta promedio por sucursal
venta_promedio_sucursal <- aggregate(
  ventas ~ sucursal,
  data = ventas,
  FUN = mean
)
cat("\nVenta promedio por sucursal:\n")
print(venta_promedio_sucursal)
#         - ticket promedio (ventas totales / clientes totales)
# Ticket promedio global:
# ventas totales / clientes totales
ticket_promedio <- sum(ventas$ventas) / sum(ventas$clientes)

cat(sprintf(
  "\nTicket promedio global: %.2f\n",
  ticket_promedio
))

#         - sucursal con mayor y menor venta
# Sucursal con mayor venta
indice_mayor <- which.max(ventas$ventas)

sucursal_mayor <- ventas[indice_mayor, ]

cat("\nSucursal con mayor venta:\n")
print(
  sucursal_mayor[
    c("sucursal", "ventas")
  ]
)

# Sucursal con menor venta
indice_menor <- which.min(ventas$ventas)

sucursal_menor <- ventas[indice_menor, ]

cat("\nSucursal con menor venta:\n")
print(
  sucursal_menor[
    c("sucursal", "ventas")
  ]
)
```

3.9 Respuesta a las preguntas

- ¿Cuántas filas y columnas tiene la tabla en cada uno de los tres formatos? ¿Fue igual en los tres? ¿Por qué?

Los archivos CSV, JSON y Excel contienen una tabla de 5 filas y 4 columnas. En los tres formatos se obtuvo la misma estructura porque representan el mismo reporte semanal de ventas, aunque provienen de sistemas diferentes. Las cuatro variables presentes son sucursal, región, ventas y clientes. El objetivo del laboratorio precisamente es comprobar que los tres formatos describen la misma información.

- ¿Qué tipo de datos se infirió para ventas y clientes? ¿Qué indicaría que salieran como texto?

Las variables ventas y clientes fueron interpretadas como datos numéricos. Esto permite realizar directamente operaciones como promedios, sumas, divisiones y agrupaciones. Si alguna de estas columnas apareciera como texto, podría indicar un problema en la calidad o formato de los datos, por ejemplo valores con símbolos, caracteres adicionales, separadores incorrectos o registros no numéricos. En ese caso sería necesario limpiar o convertir la columna antes de realizar los cálculos.

- En Python, ¿qué estructura devuelve json.load y por qué hay que "aplanarla"?

En Python, json.load() carga el contenido del archivo JSON como estructuras propias del lenguaje, normalmente listas y diccionarios. En esta práctica, después de cargar el archivo se utilizó pd.json_normalize(datos) para convertir la información a una estructura tabular tipo DataFrame. Esta transformación permite trabajar con filas y columnas de forma similar a los archivos CSV y Excel y facilita el análisis con pandas. En tu desarrollo ya utilizaste precisamente json.load() y luego pd.json_normalize().

- ¿Para qué sirve indicar sheet_name/sheet al leer Excel?

El parámetro sheet_name en Python y sheet en R permite indicar qué hoja del archivo Excel se desea leer. Esto es importante porque un archivo .xlsx puede contener varias hojas y el programa necesita saber cuál contiene los datos que se van a analizar. En esta práctica se especificó la hoja de ventas al cargar el archivo Excel tanto en Python como en R.

- ¿Qué diferencia ves entre leer en Python y leer en R (conceptos, no sintaxis)?

Conceptualmente, ambos lenguajes permiten realizar el mismo proceso: cargar los datos, convertirlos a una estructura tabular, inspeccionar sus dimensiones y tipos, y realizar cálculos sobre ellos. La principal diferencia observada es que Python centraliza gran parte del trabajo tabular mediante pandas, mientras que en R se utilizaron paquetes diferentes según el formato, como readr, jsonlite y readxl. A pesar de estas diferencias en las herramientas utilizadas, el objetivo final es el mismo: obtener una tabla estructurada que pueda ser inspeccionada y analizada.

4 · ¿Qué aprendí con esta práctica?

Con esta práctica aprendí a trabajar con un mismo conjunto de datos almacenado en diferentes formatos y a identificar que aunque CSV, JSON y Excel contienen la misma información, cada uno requiere una forma distinta de lectura y tratamiento. En Python reforcé el uso de “pandas“ para cargar los archivos CSV y Excel y comprendí mejor el uso de “json.load()” junto con “pd.json_normalize()” para transformar datos JSON en una estructura que pueda analizarse como un DataFrame. En R practiqué el uso de paquetes como “readr”, “jsonlite” y “readxl”, además de funciones como “str()”, “dim()” y “head()” para inspeccionar la estructura de los datos.

Uno de los aspectos que más me ayudó a comprender la práctica fue la revisión de los tipos de datos. Antes veía principalmente si una columna contenía números o texto, pero ahora entiendo que verificar cómo el lenguaje interpreta cada variable es importante para evitar errores posteriores en cálculos y análisis. También aprendí a crear una variable derivada, como el ticket de venta calculado mediante la relación entre

ventas y clientes, y a utilizar operaciones de agrupamiento para determinar la región con mayor venta total.

5 · Conclusiones

1. Se verificó que los archivos CSV, JSON y Excel contienen la misma información y presentan una estructura de 5 filas y 4 columnas. Además, Python y R permitieron leer e inspeccionar correctamente los datos aunque cada formato requiere funciones y procedimientos diferentes para su tratamiento.

2. El análisis permitió obtener los principales indicadores de la semana: una venta promedio por sucursal de 12 008,00, un ticket promedio de 38,07, la sucursal Guayaquil Centro con la mayor venta de 15200, la sucursal Cuenca con la menor venta de 8540 y la región Sierra con la mayor venta total de 30860. Estos resultados describen únicamente el período analizado y no permiten realizar predicciones sobre semanas futuras.

A partir de estos resultados, se puede informar a la gerencia de RetailAndina S.A. que los tres archivos analizados son consistentes y pueden utilizarse como fuentes equivalentes para este reporte semanal. Sin embargo, los resultados corresponden únicamente a las cinco sucursales y al período analizado, por lo que describen el comportamiento observado en esta semana y no deben interpretarse como una predicción del comportamiento futuro de las ventas.

6. Uso de IA

Para revisión de ortografía: https://share.gemini.google/omqV765xve2T

7· Bibliografía

pandas documentation — pandas 3.0.6 documentation. (s. f.). https://pandas.pydata.org/docs/

R: The R Project for Statistical Computing. (s. f.). https://www.r-project.org/

Read Excel files. (s. f.). https://readxl.tidyverse.org/

Ooms, J. (2025, 27 marzo). *A Simple and Robust JSON Parser and Generator for R [R package jsonlite version 2.0.0]*. https://cran.r-project.org/web/packages/jsonlite/index.html